Automated Skill Optimization for LLM Grading Systems

Research background for exam-automark grading-skill improvement

Scope. This survey treats a grading skill as a versioned, executable instruction artifact: rubric text, prompt template, schema, examples, feedback rules, evaluation scripts, and release gates. The practical question is how much of skill improvement can be automated while preserving reproducibility, auditability, and teacher review.

sci-brain run. Source set built on 2026-07-19 with the installed sci-brain `/survey` and `/download-ref` workflow. The active KB is `.knowledge/`: 14 optimization references in this report, 32 total BibTeX entries across both surveys, 32 downloaded arXiv PDFs, and 32 rendered local markdown files. No grading model calls were run.

1. Abstract

Automated skill optimization sits between prompt engineering, program synthesis, and evaluation-driven software engineering. The relevant literature does not yet use one stable vocabulary: papers speak about automatic prompt engineering, prompt optimization, language-model programs, textual gradients, reflection, self-refinement, lifelong skill libraries, and multi-agent optimization [1], [2], [3], [4], [5]. For exam-automark, the useful abstraction is an optimization loop over a frozen grading artifact. The loop proposes a change, evaluates it on a development split with rubric-level metrics, records the trace, and promotes the change only through a human-readable review gate. This report organizes automated approaches into five method families and identifies how they can inform future grading-skill work without running models in this survey.

2. Background

LLM application behavior is highly sensitive to instructions, examples, schemas, retrieval context, tool use, and output validation. Manual prompt engineering can produce strong local improvements, but it is hard to reproduce: small prompt edits are rarely tied to testable hypotheses, and improvements on one batch can regress on another batch. Grading makes this problem sharper because a "better" skill must satisfy several objectives at once: exact agreement with reference marks, low severe-error rate, stable feedback, strict JSON validity, fairness across questions and student groups, and interpretable partial-credit decisions.

The optimization literature offers a useful shift: prompts and skills can be treated as artifacts to search, compile, critique, and version. Early work such as AutoPrompt searched token triggers for masked language models [1]. Later systems such as APE, ProTeGi/APO, and OPRO use LLMs to propose or edit natural-language instructions and select candidates with task metrics [2], [3], [6]. DSPy and MIPRO generalize this from one prompt to multi-stage language-model programs [4], [7]. TextGrad and GEPA push the idea further by using natural-language feedback as an optimization signal over components of compound AI systems [5], [8].

For exam-automark, this is not simply a way to "make a prompt better." It is a way to make skill revisions empirical. A grading-skill optimizer should leave a record of what changed, which rubric failures motivated the change, which development examples improved or regressed, and why a teacher or researcher accepted the update.

3. Taxonomy Of Method Categories

3.1. Discrete Prompt And Instruction Search

These methods search over natural-language strings. A generator proposes candidate instructions, a target model applies them, and a metric ranks the results. The main strength is accessibility: the optimized artifact remains readable by humans. APE frames instructions as programs and asks an LLM to propose candidate instructions, then evaluates candidates on task examples [2]. ProTeGi/APO uses natural-language "gradients" derived from errors and edits the prompt in the opposite semantic direction [3]. OPRO treats the LLM itself as an optimizer that sees previous solutions and scores, then proposes new solutions [6].

For grading skills, this family maps naturally to prompt-template edits, rubric wording, allowed evidence types, and output-schema instructions. The core risk is overfitting to a small development set or optimizing a superficial metric while making feedback worse.

3.2. Continuous And Soft Prompt Optimization

Soft prompt methods learn embeddings or prefix vectors while keeping the base model frozen. Prefix-tuning and prompt tuning are parameter-efficient, scalable alternatives to full fine-tuning [9], [10]. They are less attractive for exam-automark's current skill work because the resulting artifact is not easily inspectable by a teacher and is model-specific. They are still

useful as a conceptual baseline: they show that prompt-like adaptation can be optimized directly when labels and model access are available.

3.3. Language-Model Program Optimizers

DSPy treats LM applications as programs made from declarative modules and optimizes those modules against a metric [4]. MIPRO extends this to multi-stage programs by optimizing both instructions and demonstrations under downstream metrics, even when module-level labels are absent [7]. DSPy Assertions adds computational constraints that can be used at compile time and inference time [11].

This is the most directly relevant family for exam-automark once grading is expressed as a pipeline: parse submission, extract evidence, apply rubric, validate schema, generate feedback, and audit high-risk cases. The optimizer can target the whole pipeline while still recording which module changed.

3.4. Textual Gradients And Reflective Evolution

TextGrad uses natural-language feedback as a form of automatic differentiation over variables in a computation graph [5]. GEPA uses reflection over trajectories, prompt mutation, and Pareto-style selection to improve compound systems with fewer rollouts than reinforcement learning baselines [8]. These methods are especially relevant when failures are easier to explain in language than to encode as scalar gradients.

For grading, textual feedback can encode why an item was wrong: hallucinated student work, ignored rubric clause, over-penalized equivalent expression, or invalid JSON. The optimizer can then propose targeted prompt edits. The danger is black-box reflection: if the optimizer's critique is not tied to concrete examples and metrics, it can create persuasive but ungrounded changes.

3.5. Self-Refinement, Agent Memory, And Skill Libraries

Self-Refine shows how a model can iteratively critique and refine its own output without additional training [12]. Reflexion stores verbal lessons from prior trials [13]. Voyager uses an ever-growing skill library of executable code, an automatic curriculum, and feedback-driven repair in Minecraft [14]. These systems are not grading optimizers, but they show a pattern exam-automark can borrow: convert successful tactics into reusable, versioned skills.

For exam-automark, the analogous artifact is a skill library of rubric-handling patterns, evidence extraction patterns, schema repair patterns, and known failure-mode mitigations. The library should be curated and tested rather than allowed to grow automatically without review.

4. Representative Work

Work	Year	Optimized artifact	Signal	Use for grading skills
AutoPrompt [1]	2020	Discrete trigger tokens for masked language models	Gradient-guided search	Historical baseline for automatic prompt construction; less suitable for readable teacher-facing skills.
Prefix-tuning [9]	2021	Continuous prefix vectors	Backpropagation on labeled data	Useful conceptually, but weak auditability for teacher review.
Prompt tuning [10]	2021	Soft prompt embeddings	Backpropagation on labeled examples	Model-specific adaptation; not ideal for repo-stored skill text.
APE [2]	2022	Natural-language instructions	Candidate generation and task scoring	Direct analogue for searching rubric prompts and output instructions.
Self-Refine [12]	2023	Generated output at inference time	Self-feedback and iterative revision	Can repair feedback or scoring rationales, but must be bounded by schema checks.
Reflexion [13]	2023	Agent memory and future behavior	Verbal reinforcement from task feedback	Useful for storing failure-mode lessons across grading trials.
ProTeGi/APO [3]	2023	Natural-language prompts	Textual gradients, beam search, bandits	Strong fit for dev-set prompt revision with severe-error metrics.
OPRO [6]	2023	Prompt or solution candidates	LLM proposes new candidates from prior scores	Simple optimizer loop, but vulnerable to metric gaming.

DSPy [4]	2023	LM pipeline modules	Compiler optimizes toward metrics	Best architectural fit for reproducible grading pipelines.
MIPRO [7]	2024	Instructions and demonstrations for LM programs	Downstream metric with surrogate optimization	Promising for multi-stage rubric/evidence/feedback pipelines.
TextGrad [5]	2024	Variables in compound AI systems	Natural-language feedback as gradient	Can turn rubric-item errors into targeted prompt updates.
GEPA [8]	2025	Prompts in compound systems	Reflective trajectory analysis and Pareto search	Relevant future direction; needs careful trace auditing in grading.

5. Comparison Table

Category	Required assets	What changes	Strengths	Risks
Discrete instruction search	Dev examples, metric, model API	Prompt text, rubric wording, examples	Readable diffs; easy to version; fast experiments [2], [3]	Overfits dev set; may optimize for score while harming explanation quality.
Soft prompt tuning	Labeled data and model training access	Embedding vectors or prefix parameters	Parameter-efficient; strong when labels are abundant [9], [10]	Opaque artifact; model lock-in; hard to inspect or merge as a skill.
Program optimizers	Pipeline modules, dev metric, validation scripts	Module prompts, demos, sometimes control flow	Optimizes end-to-end behavior; matches exam-automark architecture [4], [7]	Credit assignment is hard; costs can grow quickly.
Textual gradients	Failure traces, critique prompt, candidate evaluator	Natural-language prompt/code variables	Turns qualitative errors into actionable edits [5], [8]	Critiques may hallucinate; requires example-grounded evidence.
Reflection and skill libraries	Episode traces, success criteria, retrieval scheme	Memory, reusable skills, repair tactics	Accumulates reusable know-how; supports lifelong improvement [12], [13], [14]	Uncurated memory can amplify stale or wrong tactics.

6. Relevance To exam-automark

Immediate use. Treat every grading skill as a versioned artifact with a frozen development split, an evaluation command, and a release note. Candidate changes should be judged by exact agreement, total-score MAE, within-tolerance rate, severe-error rate, per-question deltas, schema validity, and feedback quality.

Do not automate yet. Do not let an optimizer update production skill text from private data without a recorded prompt packet, anonymized examples, human review, and held-out evaluation. Automatic proposals can draft changes; promotion must remain auditable.

The repo already has useful anchors: experiment records under `experiments/records/`, frozen skill snapshots under `experiments/skill_versions/`, prompt templates, data inventories, and Typst notes. The optimization loop can reuse those anchors:

- Build a small public or anonymized development set with rubric-item labels.
- Record each candidate skill revision as a patch against a skill snapshot.
- Run a deterministic evaluator that reports aggregate and per-item metrics.
- Generate a compact failure report showing which rubric clauses changed.
- Require a teacher or researcher to accept, reject, or edit the candidate.
- Only then freeze a new skill version and test on held-out data.

In this framing, automated skill optimization becomes less like ad hoc prompt tuning and more like CI for grading behavior. The metric is not “did the prompt look better?” but “did this version reduce validated grading errors without introducing unacceptable regressions?”

7. Open Problems

- **Reward design.** A scalar score can hide dangerous behavior. A skill that raises exact agreement but increases severe errors should not be promoted.
- **Small and noisy data.** Real course data is scarce, private, and often single-rater. Optimizers need uncertainty estimates and teacher adjudication.
- **Rubric ambiguity.** Some failures reveal ambiguous rubrics, not model errors. The optimizer should flag these rather than silently patching around them.
- **Generalization.** A prompt optimized for Physics Week 9 may not transfer to DSAA, linear algebra, or multimodal submissions.
- **Multimodal evidence.** Image quality, OCR, formula parsing, and diagram interpretation create failure modes that text-only prompt optimizers do not cover.
- **Traceability.** Every proposed change needs a source: examples, failures, metric deltas, and the exact model/tool configuration that produced the proposal.
- **Cost and reproducibility.** Search loops can consume many model calls. A reproducible framework needs budgets, cached traces, and stable evaluation commands.

8. References

Generated from the sci-brain KB BibTeX file copied from `.knowledge/references.bib`.

Bibliography

[1] T. Shin, Y. Razeghi, R. L. L. IV, E. Wallace, and S. Singh, "Eliciting Knowledge from Language Models Using Automatically Generated Prompts," pp. 4222–4235, 2020.

[2] Y. Zhou *et al.*, "Large Language Models Are Human-Level Prompt Engineers," *ArXiv*, vol. abs/2211.01910, 2022.

[3] R. Pryzant, D. Iter, J. Li, Y. Lee, C. Zhu, and M. Zeng, "Automatic Prompt Optimization with "Gradient Descent" and Beam Search," pp. 7957–7968, 2023.

[4] O. Khattab *et al.*, "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines," *ArXiv*, vol. abs/2310.03714, 2023.

[5] M. Yuksekgonul *et al.*, "TextGrad: Automatic "Differentiation" via Text," *ArXiv*, vol. abs/2406.07496, 2024.

[6] C. Yang *et al.*, "Large Language Models as Optimizers," *ArXiv*, vol. abs/2309.03409, 2023.

[7] K. Opsahl-Ong *et al.*, "Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs," *ArXiv*, vol. abs/2406.11695, 2024.

[8] L. A. Agrawal *et al.*, "GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning," *ArXiv*, vol. abs/2507.19457, 2025.

[9] X. L. Li and P. Liang, "Prefix-Tuning: Optimizing Continuous Prompts for Generation," *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 4582–4597, 2021.

[10] B. Lester, R. Al-Rfou, and N. Constant, "The Power of Scale for Parameter-Efficient Prompt Tuning," pp. 3045–3059, 2021.

[11] A. Singhvi *et al.*, "DSPy Assertions: Computational Constraints for Self-Refining Language Model Pipelines," *ArXiv*, vol. abs/2312.13382, 2023.

[12] A. Madaan *et al.*, "Self-Refine: Iterative Refinement with Self-Feedback," *ArXiv*, vol. abs/2303.17651, 2023.

[13] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems 36*, 2023.

[14] G. Wang *et al.*, "Voyager: An Open-Ended Embodied Agent with Large Language Models," *ArXiv*, vol. abs/2305.16291, 2023.